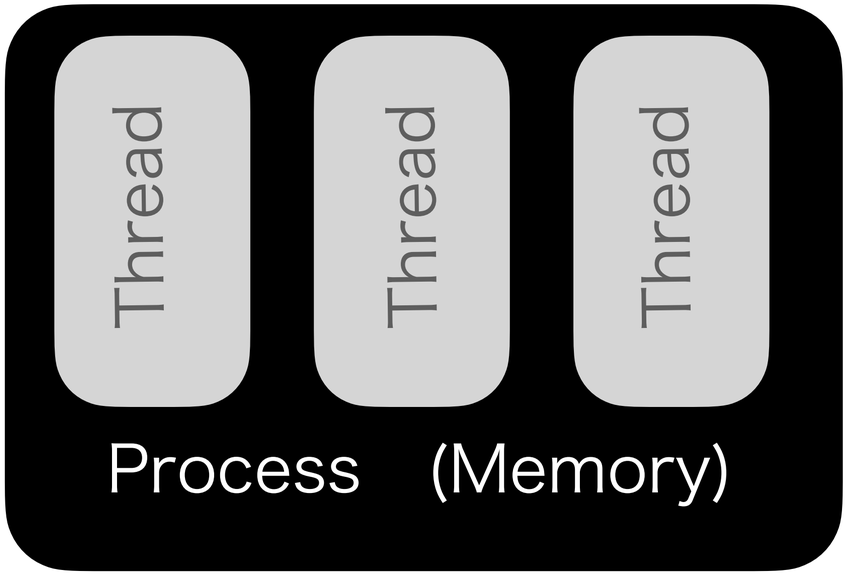
並列化の方法：OpenMPとMPI の超入門

大阪大学D3センター
吉野　元

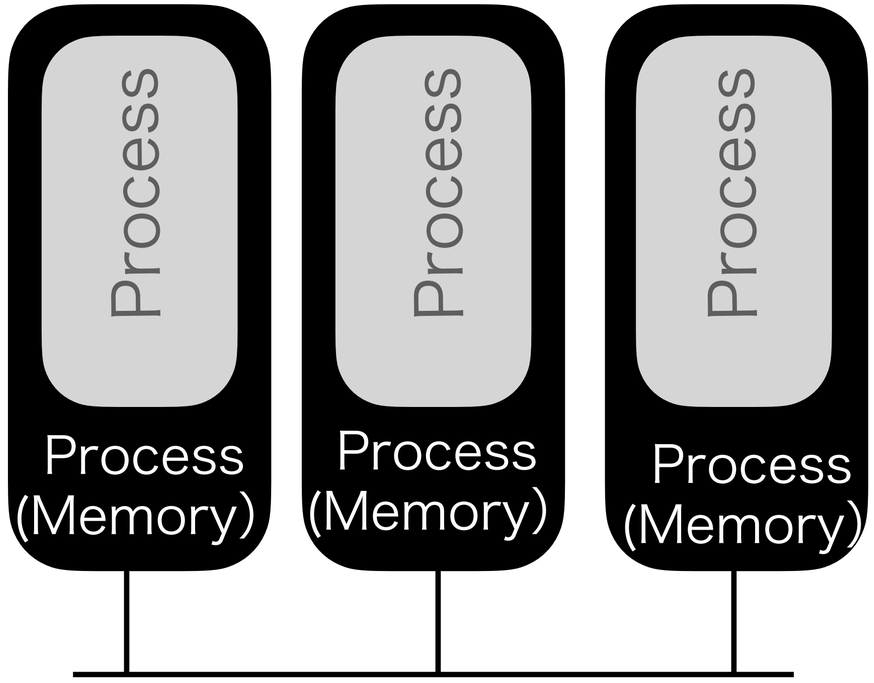
OpenMP/MPIとは？

OpenMP/自動並列化



ノードをまたいだ並列化はできない
(富岳のA64FXでは48並列まで)

MPI (Message Passing Interface)

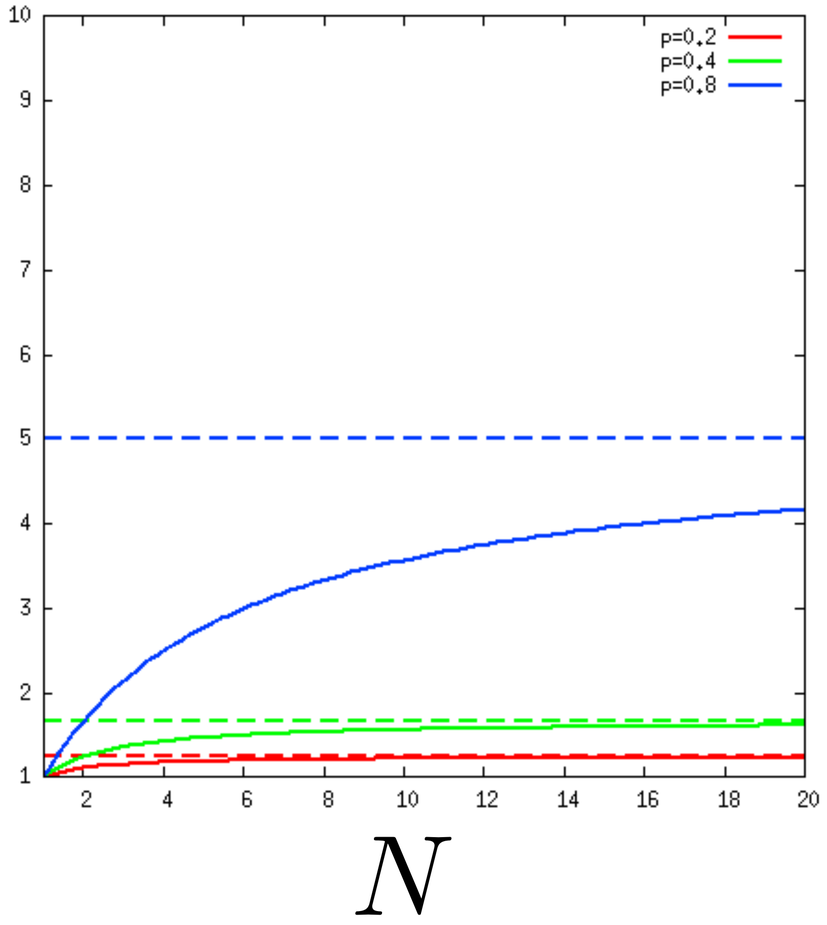


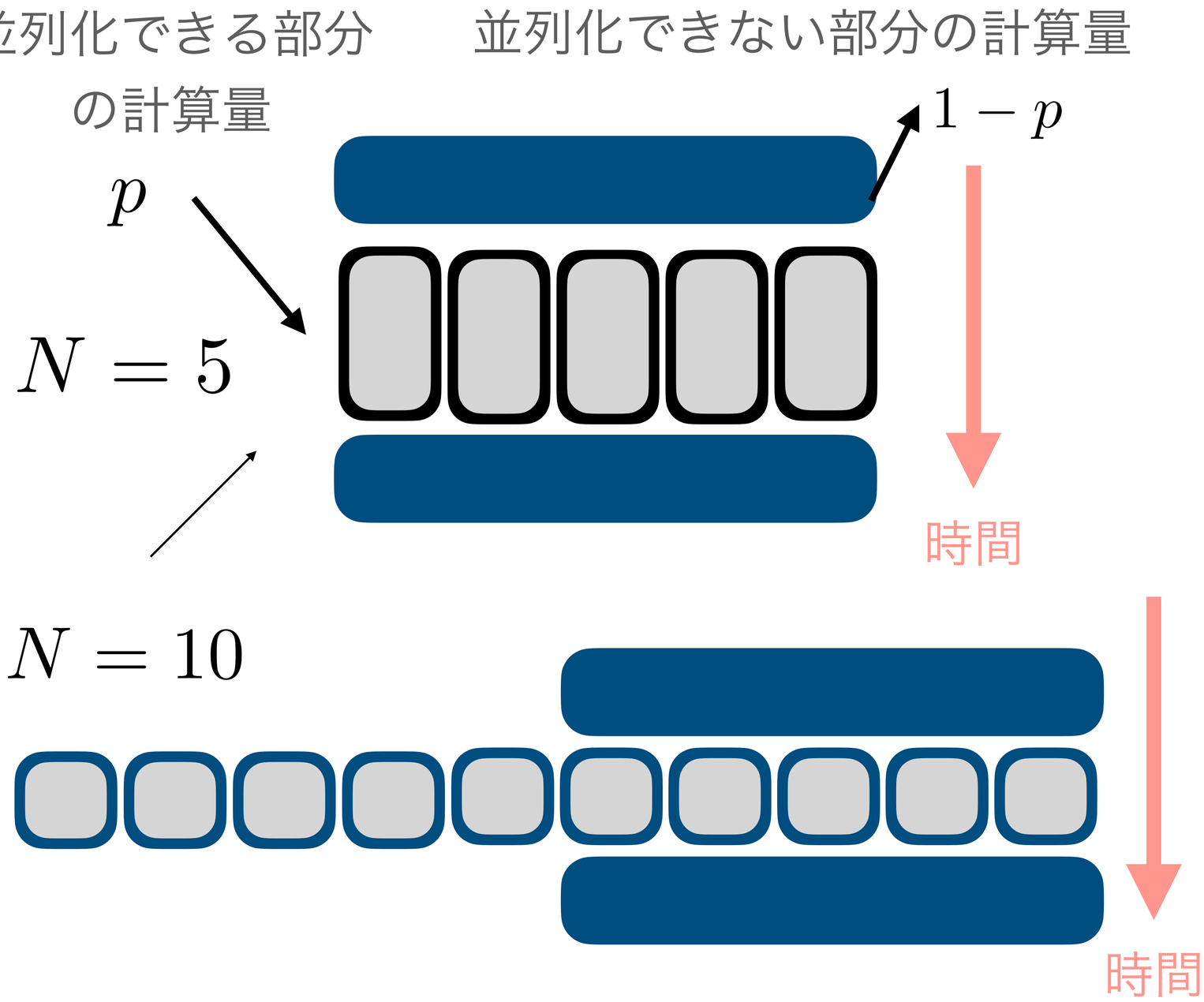
プロセス間通信

OpenMPのプロセス：一つのノード内の複数のコア上に一つずつスレッド(Thread)を作って並列に計算する。メモリーは共有。

Amdahlの法則

並列化効率= $\dfrac{1}{1-p+\frac{p}{N}}$





サンプルコード、ジョブスクリプトの取得とコンパイル

1. サンプルコードとジョブスクリプトを取得: discord の「資料置き場」に置いてあるsample_openmp_mpi.tarをダウンロード

2.ブラウザでFugaku OnDemandにアクセス　https://ondemand.fugaku.r-ccs.riken.jp/pun/sys/dashboard







3. クリックするとホームディレクトリのファイル一覧。ここにsample_omp_mpi.tarをアップロード

4. クリックするとlinuxのシェルが立ち上がる。

1)解凍作業： tar -xvf sample_omp_mpi.tar

2) openMPのサンプルのコンパイル:
cd ./sample_omp_mpi/ompでompに移り、
そこでsh compile.sh

3) MPIのサンプルのコンパイル: cd ../MPIでMPIに移り、そこでもsh compile.sh

以上でサンプルプログラムをコンパイルした実行ファイルが出来たはず。



あるいは

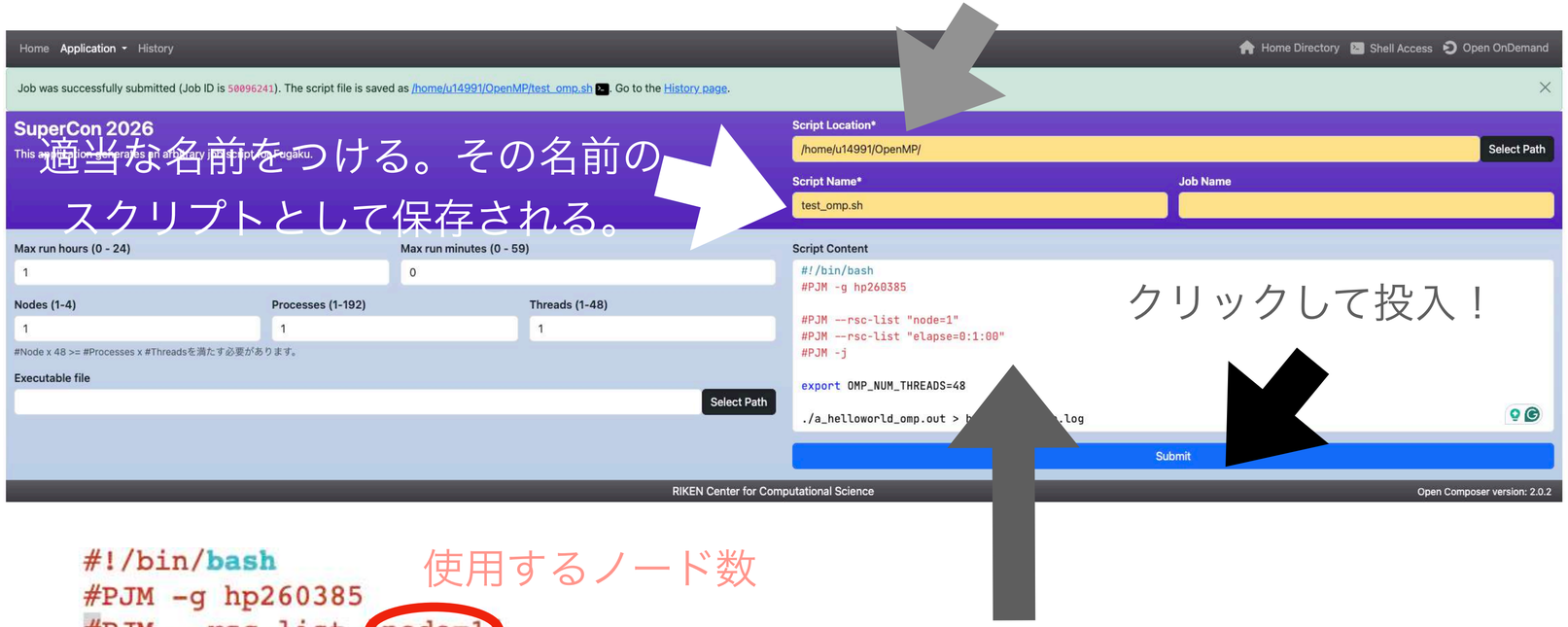


で確認しよう。

OpenMP超入門

このスクリプトの想定するdirectoryである /home/u…/OpenMP を入れる





/OpenMP/helloworld_omp.sh

```
#!/bin/bash
#PJM -g hp260385
#PJM --rsc-list "node=1"
#PJM --rsc-list "elapse=0:1:00"
#PJM -j

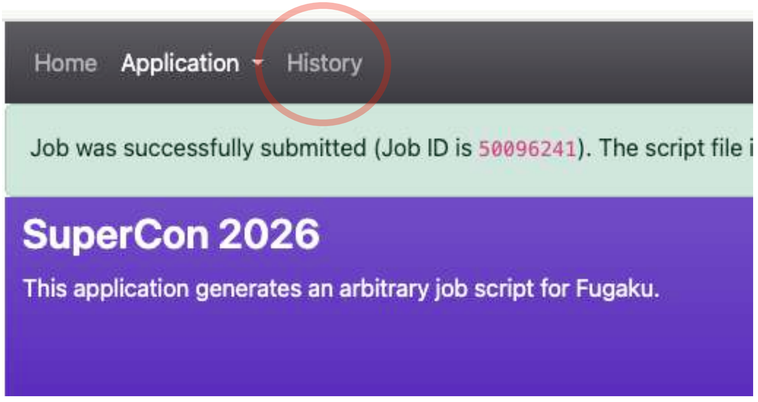
export OMP_NUM_THREADS=48

./a_helloworld_omp.out > helloworld_omp.log
```

使用するノード数

OpenMPのスレッド数

テキストをcut & pasteで貼り付け





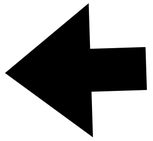
終わっている！

(1) hello world

/OpenMP/helloworld_omp.c

```c
# include <stdio.h>
# include <omp.h>

int main(){
  #pragma omp parallel
    {
    printf("HELLO WORLD mythread = %d\n",omp_get_thread_num());
    }
    return 0;
}
```



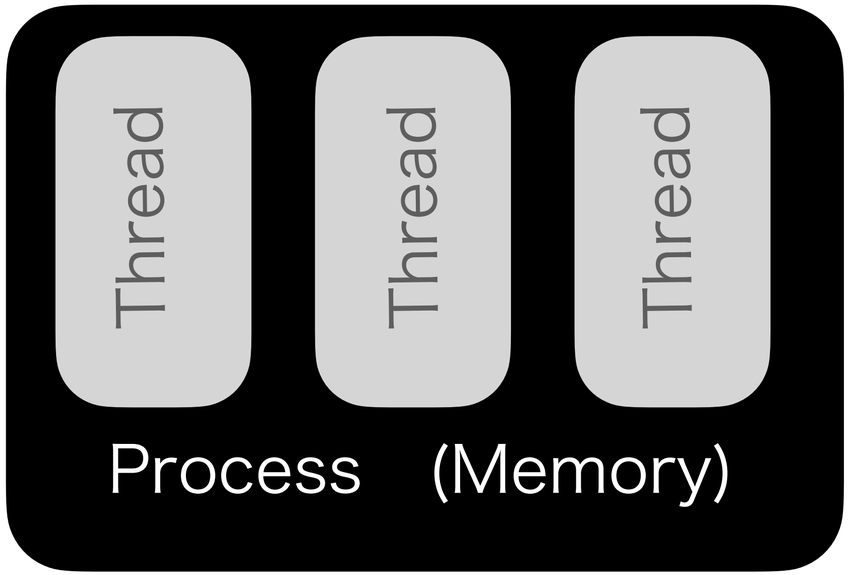
このfor ループが並列化される

結果

/OpenMP/helloworld_omp.log

```
[login6$ pwd
 /home/u14991/OpenMP
[login6$ cat helloworld_omp.log
 HELLO WORLD mythread = 0
 HELLO WORLD mythread = 2
 HELLO WORLD mythread = 3
 HELLO WORLD mythread = 1
 HELLO WORLD mythread = 4
 HELLO WORLD mythread = 5
 HELLO WORLD mythread = 6
 HELLO WORLD mythread = 32
 HELLO WORLD mythread = 9
 HELLO WORLD mythread = 11
 HELLO WORLD mythread = 16
 HELLO WORLD mythread = 13
 HELLO WORLD mythread = 34
 HELLO WORLD mythread = 8
 HELLO WORLD mythread = 17
 HELLO WORLD mythread = 33
 HELLO WORLD mythread = 7
 HELLO WORLD mythread = 10
 HELLO WORLD mythread = 18
 HELLO WORLD mythread = 19
 HELLO WORLD mythread = 12
 HELLO WORLD mythread = 14
 HELLO WORLD mythread = 44
 HELLO WORLD mythread = 28
 HELLO WORLD mythread = 15
 HELLO WORLD mythread = 24
 HELLO WORLD mythread = 20
 HELLO WORLD mythread = 21
 HELLO WORLD mythread = 35
 HELLO WORLD mythread = 36
 HELLO WORLD mythread = 40
 HELLO WORLD mythread = 22
 HELLO WORLD mythread = 23
 HELLO WORLD mythread = 26
 HELLO WORLD mythread = 29
 HELLO WORLD mythread = 25
 HELLO WORLD mythread = 39
 HELLO WORLD mythread = 38
 HELLO WORLD mythread = 37
 HELLO WORLD mythread = 43
 HELLO WORLD mythread = 42
 HELLO WORLD mythread = 41
 HELLO WORLD mythread = 46
 HELLO WORLD mythread = 45
 HELLO WORLD mythread = 47
 HELLO WORLD mythread = 31
 HELLO WORLD mythread = 30
 HELLO WORLD mythread = 27
```

スレッドでfor ループの中身を分担



(2) vector_sum

```c
# include <stdio.h>
# include <math.h>
# include <omp.h>


int main(){
  const int num_size=1000;
  double a[num_size],b[num_size],c[num_size];
  int i;
  double st, en;

  for(i=0;i<num_size;i++){a[i]=rand()/pow(2,31);};
  for(i=0;i<num_size;i++){b[i]=rand()/pow(2,31);};

  st = omp_get_wtime();

  #pragma omp parallel for
  for(i=0;i<num_size;i++){
    c[i]=a[i]+b[i];
  };

  en = omp_get_wtime();
  printf("#Elapsed time in second is: %f\n", en-st);
  return 0;
}
```

(3)vector_inner_product

総和の計算

```
# include <stdio.h>
# include <omp.h>
# include <math.h>

int main(){
  const int num_size=100000;
  double a[num_size],b[num_size];
  int i;
  double st, en;
  double s;

  for(i=0;i<num_size;i++){a[i]=rand()/pow(2,31);};
  for(i=0;i<num_size;i++){b[i]=rand()/pow(2,31);};

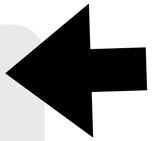
  st = omp_get_wtime();

  s=0.0;
#pragma omp parallel for reduction(+:s)
  for(i=0;i<num_size;i++){
    s=s+a[i]*b[i];
  };

#pragma omp barrier

  en = omp_get_wtime();
  printf("#Elapsed time in second is: %f\n", en-st);

    return 0;
}
```



変数sに+: 総和をためてゆく

（注）+以外の演算もあり 乗算* 減算-, 最大値の抽出max, 最小値の抽出min, ビット演算も

Private変数/Shared変数

デフォルトではShared変数(スレッド間で共有)、各スレッドで持っていたい変数はPrivate変数として指定する必要あり。ただし、ループの変数は自動的にPrivate変数としてみなされている。またパラレルスコープ中で宣言されたものはPrivateとみなされている。

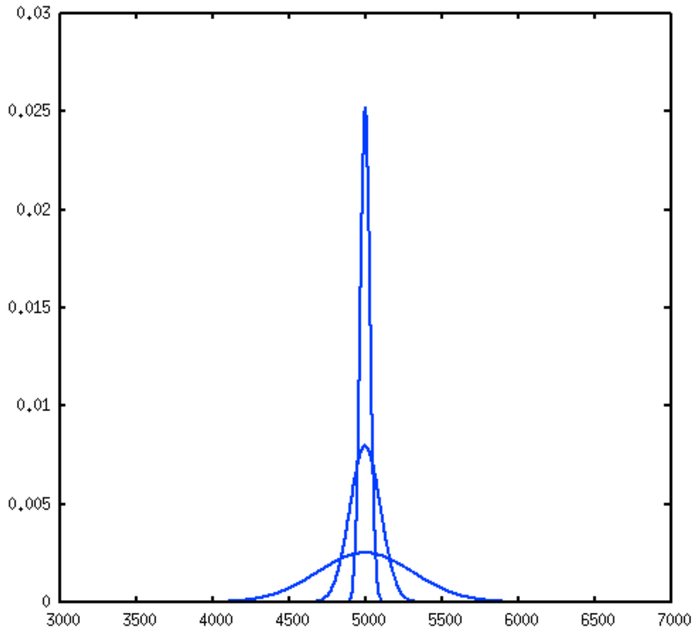
スレッドごとに
tmpを書き換えてしまうので
ダメ

```
#pragma omp for
for (i=0; i< 1000; i++) {
  tmp = some_function(i);
  a[i] = tmp;
 }
```

```
#pragma omp for private(tmp)
for (i=0; i< 1000; i++) {
  tmp = some_function(i);
  a[i] = tmp;
 }
```

(4) diffusion

1次元の拡散




```
# include <stdio.h>
# include <omp.h>

int main(){
  const int num_size=10000;
  const int no_time=20000;

  double p0[num_size],p[num_size];
  double r;

  int ip[num_size],im[num_size];

  int i,it;
  double st, en;

  r=0.5;

    for(i=0;i<num_size;i++){
      ip[i]=(i+1) % num_size;
      im[i]=(i-1+num_size) % num_size;
    };

    for(i=0;i<num_size;i++){
      p0[i]=0.0;
    };

  p0[  num_size/2]=0.5;
  p0[  num_size/2+1]=0.5;

  st = omp_get_wtime();

#pragma omp parallel private(it)
  {

  for(it=0; it < no_time; it++){

#pragma omp for
    for(i=0;i<num_size;i++){
      p[i]=r*p0[ip[i]]+(1.0-r)*p0[im[i]];
    };

#pragma omp for
    for(i=0;i<num_size;i++){
      p0[i]=p[i];
    };
  };
  };

en = omp_get_wtime();
printf("#Elapsed time in second is: %f\n", en-st);

for(i=0;i<num_size;i++){
  printf("%d %f\n",i,p[i]);
 };

return 0;
}
```

スレッド生成するのは時間を消費するので
it のループの外側で並列化領域をとった。

変数itはPrivate変数とした

(5) reaction_diffusion

反応拡散

$A + B \rightarrow C$

```
#pragma omp for
      for(i=0;i<num_size;i++){
        p[i]=r*p0[ip[i]]+(1.0-r)*p0[im[i]];
      };
```

↓

ループ内の計算量が増えた

```
#pragma omp for
      for(i=0;i<num_size;i++){
        pA[i]=rA*pA0[ip[i]]+(1.0-rA)*pA0[im[i]]-alpha*pA0[i]*pB0[i];
        pB[i]=rB*pB0[ip[i]]+(1.0-rB)*pB0[im[i]]-alpha*pA0[i]*pB0[i];
        pC[i]=rC*pC0[ip[i]]+(1.0-rC)*pC0[im[i]]+alpha*pA0[i]*pB0[i];
      };
```

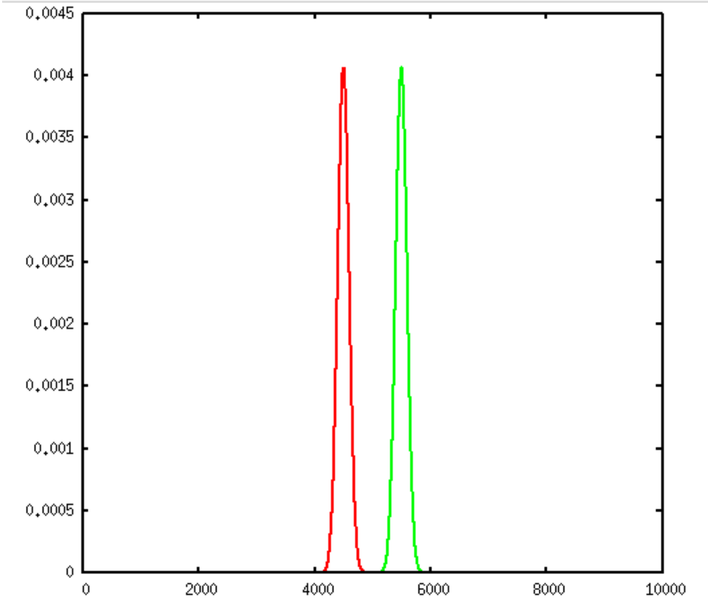
/OpenMP/reaction_diffusion_omp.sh

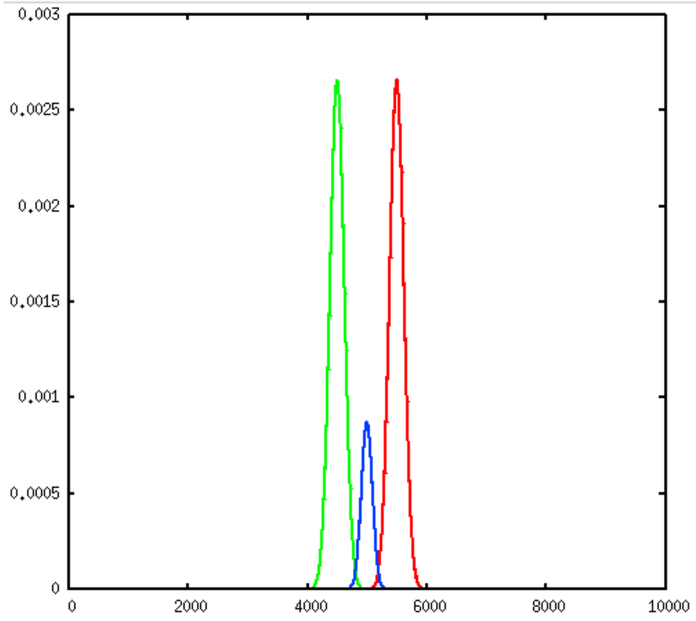
```
#!/bin/bash
#PJM -g hp260385
#PJM --rsc-list "node=1"
#PJM --rsc-list "elapse=0:1:00"
#PJM -j

export OMP_NUM_THREADS=48

logfile=log_reaction_diffusion_omp_$OMP_NUM_THREADS.file
./a_reaction_diffusion_omp.out > $logfile
echo '#NUM_THREADS='$OMP_NUM_THREADS >> $logfile
```

スレッド数を1,2,4,8,16,32などと変えて
並列化効率を確かめてみよう！





MPI超入門

(1) hello world

/MPI/hello.c

```
login5$ cat hello.c
#include "mpi.h"
#include <stdio.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

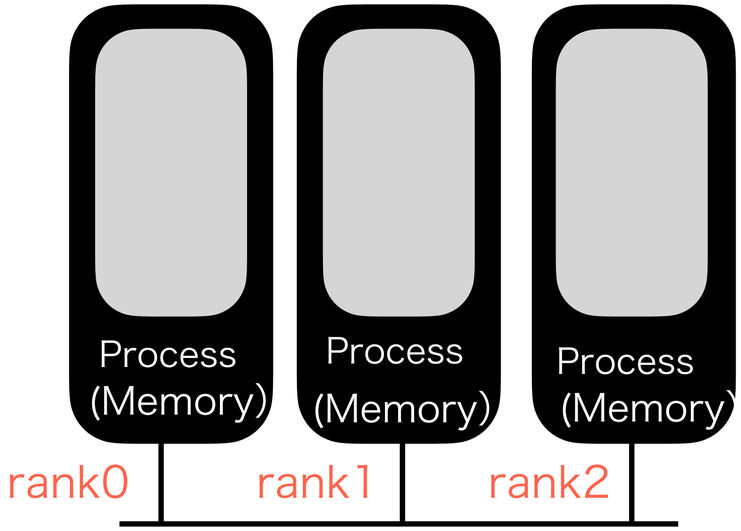
  printf ("Hello World %d %d \n", myid, n_procs);

  MPI_Finalize();
}
```

MPI_COMM_WORLD: コミュニケータ、プロセスの集合を表す。

&stat: メッセージに関する補足情報が受け取れる

各プロセスが同じプログラムを実行している。



/MPI/hello.sh

```
#!/bin/bash
#PJM -g hp260385
#PJM --rsc-list "elapse=0:1:00"
#PJM -L "node=2"
#PJM --mpi "proc=96"
#PJM --mpi "max-proc-per-node=48"
#PJM -j

mpiexec  -std-proc hello.file ./hello.out
```

2ノードで計算

合計96プロセス

48プロセス/ノード

標準出力をファイルに書き出している

結果

```
[login6$ ls hello.file.*
hello.file.1.0   hello.file.1.19  hello.file.1.29  hello.file.1.39  hello.file.1.49  hello.file.1.59  hello.file.1.69  hello.file.1.79  hello.file.1.89
hello.file.1.1   hello.file.1.2   hello.file.1.3   hello.file.1.4   hello.file.1.5   hello.file.1.6   hello.file.1.7   hello.file.1.8   hello.file.1.9
hello.file.1.10  hello.file.1.20  hello.file.1.30  hello.file.1.40  hello.file.1.50  hello.file.1.60  hello.file.1.70  hello.file.1.80  hello.file.1.90
hello.file.1.11  hello.file.1.21  hello.file.1.31  hello.file.1.41  hello.file.1.51  hello.file.1.61  hello.file.1.71  hello.file.1.81  hello.file.1.91
hello.file.1.12  hello.file.1.22  hello.file.1.32  hello.file.1.42  hello.file.1.52  hello.file.1.62  hello.file.1.72  hello.file.1.82  hello.file.1.92
hello.file.1.13  hello.file.1.23  hello.file.1.33  hello.file.1.43  hello.file.1.53  hello.file.1.63  hello.file.1.73  hello.file.1.83  hello.file.1.93
hello.file.1.14  hello.file.1.24  hello.file.1.34  hello.file.1.44  hello.file.1.54  hello.file.1.64  hello.file.1.74  hello.file.1.84  hello.file.1.94
hello.file.1.15  hello.file.1.25  hello.file.1.35  hello.file.1.45  hello.file.1.55  hello.file.1.65  hello.file.1.75  hello.file.1.85  hello.file.1.95
hello.file.1.16  hello.file.1.26  hello.file.1.36  hello.file.1.46  hello.file.1.56  hello.file.1.66  hello.file.1.76  hello.file.1.86
hello.file.1.17  hello.file.1.27  hello.file.1.37  hello.file.1.47  hello.file.1.57  hello.file.1.67  hello.file.1.77  hello.file.1.87
hello.file.1.18  hello.file.1.28  hello.file.1.38  hello.file.1.48  hello.file.1.58  hello.file.1.68  hello.file.1.78  hello.file.1.88
```

```
[login6$ cat hello.file.1.27
 Hello World 27 96
```

MPI_SENDとMPI_RECV

rank1からrank0にデータを送りたい。

/MPI/send_recv.c

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Status status;

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

  int A,B;
  A=myid;
  B=0;

  int itag=0;
  if (myid==1){
    MPI_Send(&A,1,MPI_INT,0,itag,MPI_COMM_WORLD);
  }else{
    if (myid==0){
      MPI_Recv(&B,1,MPI_INT,1,itag,MPI_COMM_WORLD, &status);
    };
  };
  printf ("A= %d B= %d at %d \n", A, B, myid);

  MPI_Finalize();
}
```

データが届くまで先には進まない（ブロッキング)

結果

```
A= 1 B= 0 at 1
A= 0 B= 1 at 0
A= 5 B= 0 at 5
A= 2 B= 0 at 2
A= 9 B= 0 at 9
A= 3 B= 0 at 3
A= 11 B= 0 at 11
A= 7 B= 0 at 7
A= 4 B= 0 at 4
A= 6 B= 0 at 6
A= 8 B= 0 at 8
A= 10 B= 0 at 10
```

MPI_ISENDとMPI_IRECV

rank1からrank0にデータを送りたい。

/MPI/isend_irecv.c

非ブロッキング通信版
こちらを推奨

```c
#include "mpi.h"
#include <stdio.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Status status;
  MPI_Request request; // 非ブロッキング通信のステータス管理用

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

  int A,B;
  A=myid;
  B=0;

  int itag=0;
  if (myid==1){
    // 非ブロッキング送信の開始
    MPI_Isend(&A, 1, MPI_INT, 0, itag, MPI_COMM_WORLD, &request);
    // 送信完了を待機
    MPI_Wait(&request, &status);
  }else{
    if (myid==0){
      // 非ブロッキング受信の開始
      MPI_Irecv(&B, 1, MPI_INT, 1, itag, MPI_COMM_WORLD, &request);
      // 受信完了（Bにデータが格納されるの）を待機
      MPI_Wait(&request, &status);
    };
  };

  printf ("A= %d B= %d at %d \n", A, B, myid);
  MPI_Finalize();
}
```

結果

```
A= 1 B= 0 at 1
A= 0 B= 1 at 0
A= 5 B= 0 at 5
A= 2 B= 0 at 2
A= 9 B= 0 at 9
A= 3 B= 0 at 3
A= 11 B= 0 at 11
A= 7 B= 0 at 7
A= 4 B= 0 at 4
A= 6 B= 0 at 6
A= 8 B= 0 at 8
A= 10 B= 0 at 10
```

MPI_Bcast

集団通信

/MPI/bcast.c

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Status status;

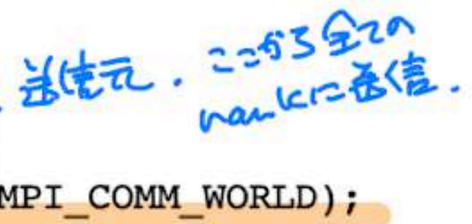
  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

  int A,B;
  A=myid;
  B=A;

  MPI_Bcast(&A,1,MPI_INT,3,MPI_COMM_WORLD);

  printf ("A= %d B= %d at %d \n", A, B, myid);

  MPI_Finalize();
}
```



結果

```
A= 3 B= 6 at 6
A= 3 B= 2 at 2
A= 3 B= 9 at 9
A= 3 B= 0 at 0
A= 3 B= 10 at 10
A= 3 B= 1 at 1
A= 3 B= 4 at 4
A= 3 B= 7 at 7
A= 3 B= 8 at 8
A= 3 B= 5 at 5
A= 3 B= 3 at 3
A= 3 B= 11 at 11
```

MPI_Reduce

集団通信

/MPI/reduce.c

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Status status;

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

  int A,B;
  A=myid;
  B=0;

  MPI_Reduce(&A,&B,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);

  printf ("A= %d B= %d at %d \n", A, B, myid);

  MPI_Finalize();
}
```



演算は他に，MPI_PROD(積), MPI_MAX, MPI_MIN, MPI_LAND(論理積)など

結果

```
A= 5 B= 0 at 5
A= 3 B= 0 at 3
A= 9 B= 0 at 9
A= 0 B= 66 at 0
A= 4 B= 0 at 4
A= 2 B= 0 at 2
A= 11 B= 0 at 11
A= 8 B= 0 at 8
A= 7 B= 0 at 7
A= 1 B= 0 at 1
A= 6 B= 0 at 6
A= 10 B= 0 at 10
```

参考： MPI_AllReduce(&A, &B, MPI_INT,MPI_SUM,MPI_COMM_WORLD)

一つのrankに集めるのではなく、同じ集めた結果を全部のrankに送る。

MPI_Gather

集団通信

/MPI/gather.c

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
  int  myid, n_procs;
  MPI_Status status;

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD,&n_procs);
  MPI_Comm_rank(MPI_COMM_WORLD,&myid);

  int A;
  A=myid;

  int *B;
  B=(int *)malloc(n_procs*sizeof(int));

  MPI_Gather(&A, 1, MPI_INT, B, 1, MPI_INT, 0, MPI_COMM_WORLD);

  if (myid==0){
    int i;
    for (i=0;i< n_procs;i++){
      printf ("B[%d]= %d\n", i,B[i]);
    };
  };

  MPI_Finalize();
}
```



結果

```
B[0]= 0
B[1]= 1
B[2]= 2
B[3]= 3
B[4]= 4
B[5]= 5
B[6]= 6
B[7]= 7
B[8]= 8
B[9]= 9
B[10]= 10
B[11]= 11
```

MPI_Barrier

MPI_Barrier(MPI_COMM_WORLD);　すべてのプロセスがここで待ち合わせ

OpenMPか？MPIか？両方か？

Supercon2026の本番で使えるは4ノード x 48コア/ノード= 192コア

0.OpenMPもMPIも使わない。	せめてシェルスクリプト（ジョブスクリプト）で並列化してはどうでしょう
1. OpenMPのみ	OpenMPによる並列化は1ノード(48スレッド）まで。 あとはシェルスクリプトによる並列化でも。
2. MPIのみ	４ノードで192プロセスの並列化まで可能。
3. OpenMP+MPI	富岳のA64FXのハードウェアに基づく推奨パターンは 1ノードを4 プロセス(MPI) x 12 スレッド (OpenMP)で使うパターン。 例えば、４ノードあるので16プロセス(MPI)x12スレッド(OpenMP)。

（参考）MPI プログラミングを全くしなくてもMPIを独立プロセスを立てる道具として活用できる：run-sc26-sample-omp.sh

OpenMP+MPIの場合のジョブスクリプトの例

```
#!/bin/bash
#PJM -g hp260385
#PJM --rsc-list "elapse=0:12:00"
#PJM -L "node=4"
#PJM --mpi "proc=16"
#PJM --mpi "max-proc-per-node=4"
#PJM -j

export OMP_NUM_THREADS=12

mpiexec -n 16 ./a.out 1
mpiexec -n 16 ./a.out 2
mpiexec -n 16 ./a.out 3
mpiexec -n 16 ./a.out 4
mpiexec -n 16 ./a.out 5
```

使用するノード数

プロセスの総数

1ノードあたりのプロセス数

OpenMPのスレッド数/プロセス

参考

discordにアップされる本戦問題のサンプルプログラム、サンプルジョブスクリプト
なども参考に。

富岳のAIチャット	https://fugaku.zendesk.com/hc/ja/p/askdona
富岳のポータルサイト	https://www.fugaku.r-ccs.riken.jp/ 以下の「ドキュメント」に各種資料